

ENGINEERING ELECTIVE • CLOUD
TRACK

Linux Shell Scripting

Automate Linux with Bash Scripts

~2h 30m / Lecture by: Imran Sayyed

```
imran@ubuntu:~$ cat hello.sh
#!/bin/bash
echo "Hello, Cloud!"

imran@ubuntu:~$ ./hello.sh
Hello, Cloud!
```

What we cover today

01

Scripts

shebang, chmod, run

02

Variables

declare, input, output

03

Conditions

if, elif, nested

04

Loops

for, while, automate

THE CASE FOR SCRIPTING

Why automation matters

BY HAND

Type it again, every time

Slow, easy to forget a step, and mistakes creep in.

WITH A SCRIPT

Write once, run anytime

Fast, repeatable, and the same result on every server.

COMMANDS IN A FILE

What is shell scripting?



A shell script is a text file of commands the shell runs top to bottom.

WHERE SCRIPTS LIVE

Real world uses

Backups

Copy data to safe storage on a schedule.

Log Analysis

Scan logs for errors and patterns.

Monitoring

Check disk, memory, and services.

Deployment

Ship and restart apps in one command.

THE DEFAULT SHELL

What is Bash?

Bash is the most common Linux shell. It reads your commands and runs your scripts.

Bash = Bourne Again SHell

Pre-installed on most distributions

Runs files that start with `#!/bin/bash`

FROM IDEA TO OUTPUT

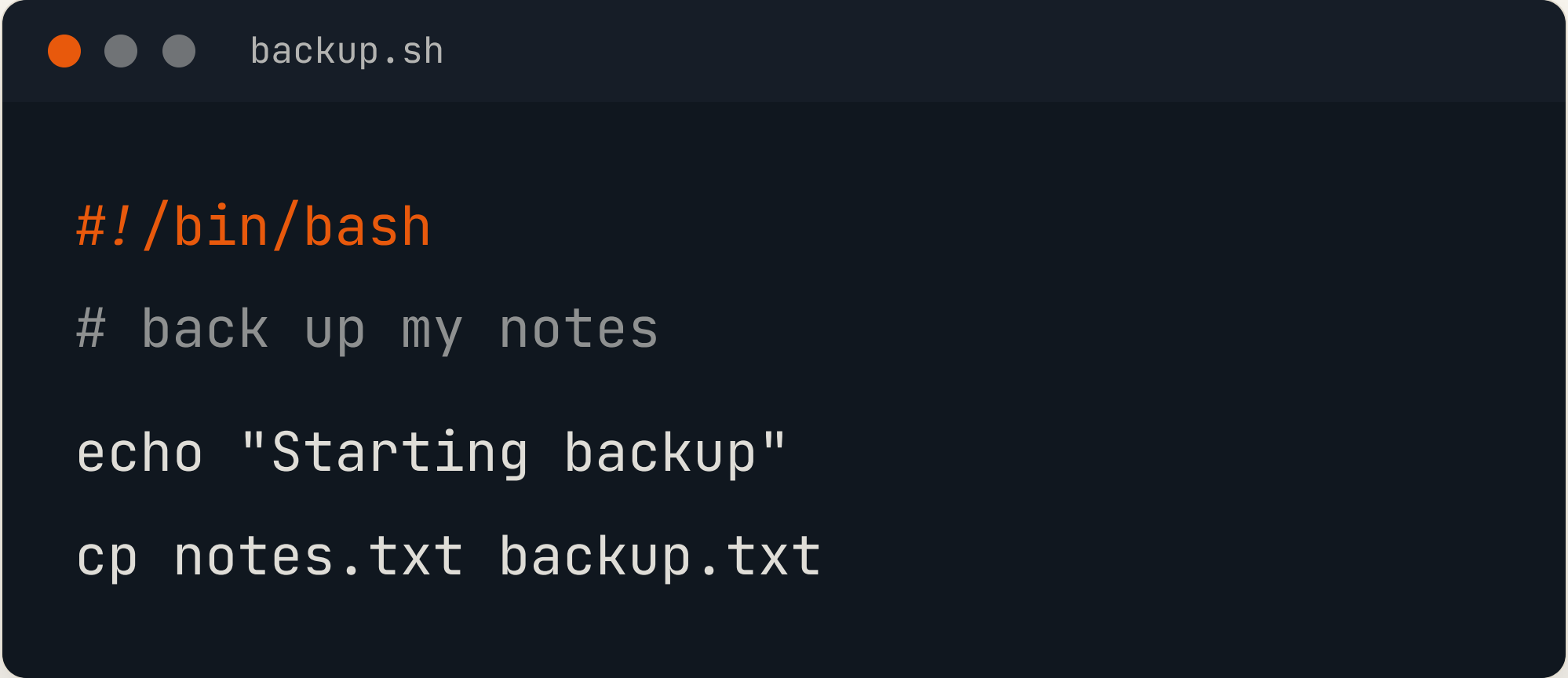
Shell script lifecycle



ANATOMY

Structure of a script

Three simple parts: the shebang, optional comments, then the commands.



```
#!/bin/bash  
# back up my notes  
  
echo "Starting backup"  
cp notes.txt backup.txt
```


THE FIRST LINE

The shebang

```
#!/bin/bash
```

The shebang tells Linux **which program** should run the file.

It must be the very first line, with no spaces before it.

HELLO, WORLD

Your first script

Open an editor, add the shebang, and print a message with **echo**.

hello.sh

```
#!/bin/bash
```

```
echo "Hello, World"
```

```
echo "My first script"
```

EXECUTE IT

Running scripts

The `./` tells Linux to run the script in the current folder.

A terminal window with a dark blue background and light gray text. The window title bar shows three colored circles (orange, gray, gray) and the text 'imran@ubuntu: ~'. The terminal content shows two commands and their outputs. The first command is './hello.sh' which outputs 'Hello, World'. The second command is 'bash hello.sh' which also outputs 'Hello, World'.

```
imran@ubuntu: ~  
  
imran@ubuntu:~$ ./hello.sh  
Hello, World  
  
imran@ubuntu:~$ bash hello.sh  
Hello, World
```

MAKE IT EXECUTABLE

chmod +x

New file

cannot run



+x

now runs

```
$ chmod +x hello.sh
```

```
$ ./hello.sh
```

```
Hello, World
```

STORING VALUES

Variables

name

"Imran"

a labelled box that holds a value

A variable is a **name** that points to a value you can reuse.

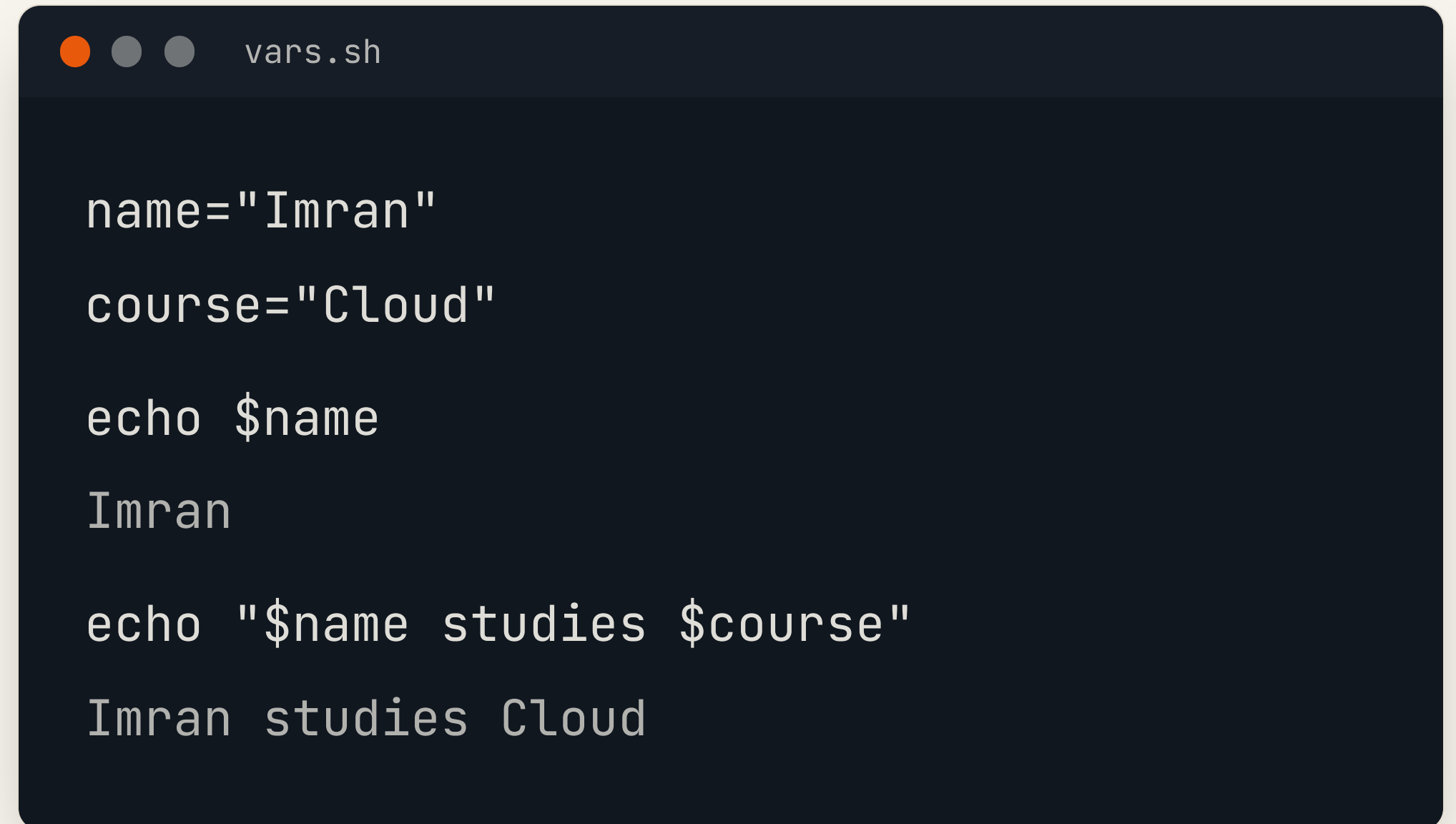
Set it once, then use it many times in your script.

SET AND USE

Declaring variables

No spaces around **=**.

Read a value back with **\$**.

A terminal window with a dark blue background and white text. The title bar at the top shows three window control buttons (red, yellow, green) and the filename 'vars.sh'. The terminal contains a shell script with four lines: 'name="Imran"', 'course="Cloud"', 'echo \$name', and 'echo "\$name studies \$course"'. The output of the script is shown on the next line: 'Imran' and 'Imran studies Cloud'.

```
vars.sh

name="Imran"
course="Cloud"

echo $name
Imran

echo "$name studies $course"
Imran studies Cloud
```

VALID NAMES

Naming rules

ALLOWED

user_name

count1

PATH

NOT ALLOWED

1count

user name

user-name

WHERE IT LIVES

Variable scope

Local

Lives only inside the script that created it.

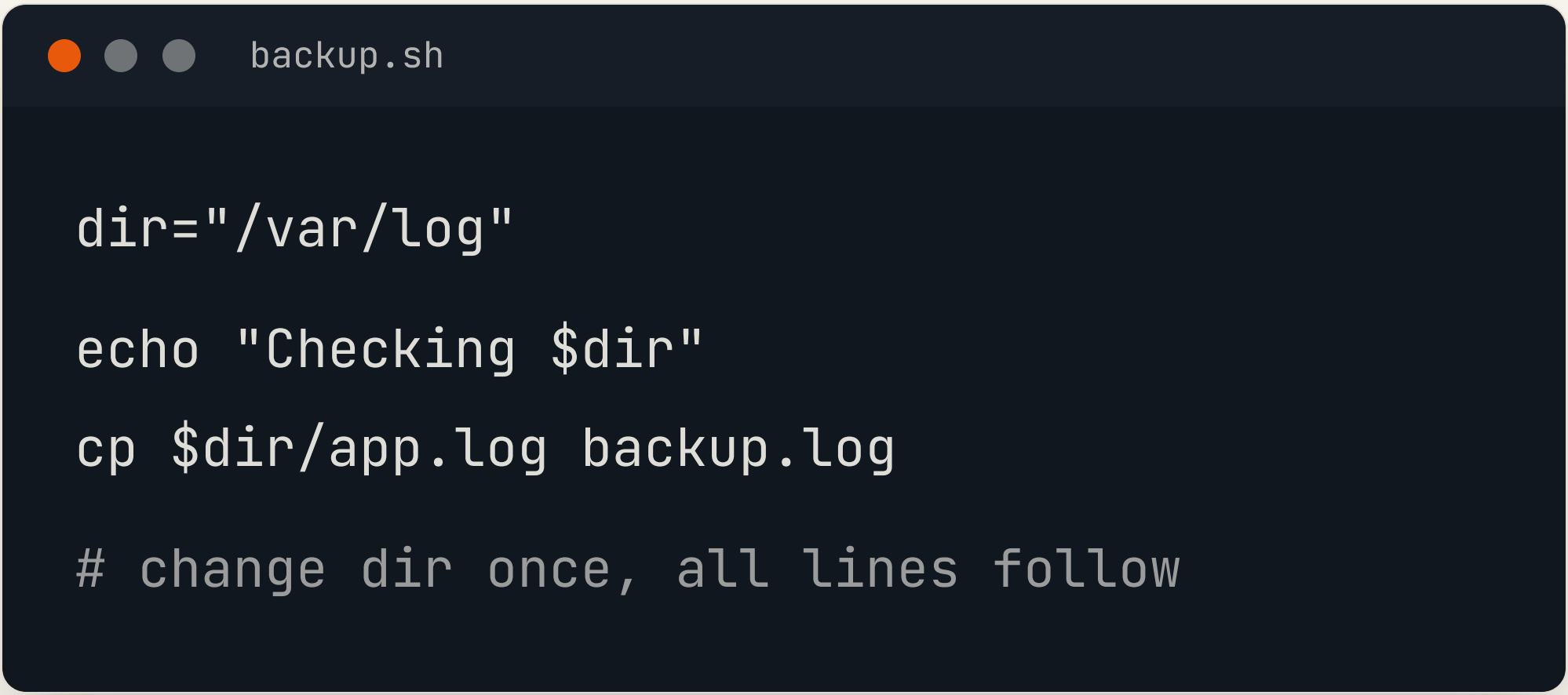
Environment

Shared with other programs using export.

PUT IT TO WORK

Practical examples

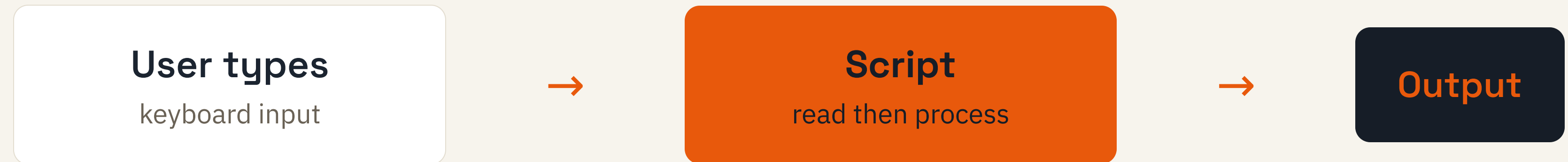
Store a path once, then reuse it across many commands.

A terminal window with a dark background and light-colored text. The title bar at the top shows three window control buttons (red, yellow, green) and the filename 'backup.sh'. The script content is as follows:

```
dir="/var/log"
echo "Checking $dir"
cp $dir/app.log backup.log
# change dir once, all lines follow
```

TALKING TO THE USER

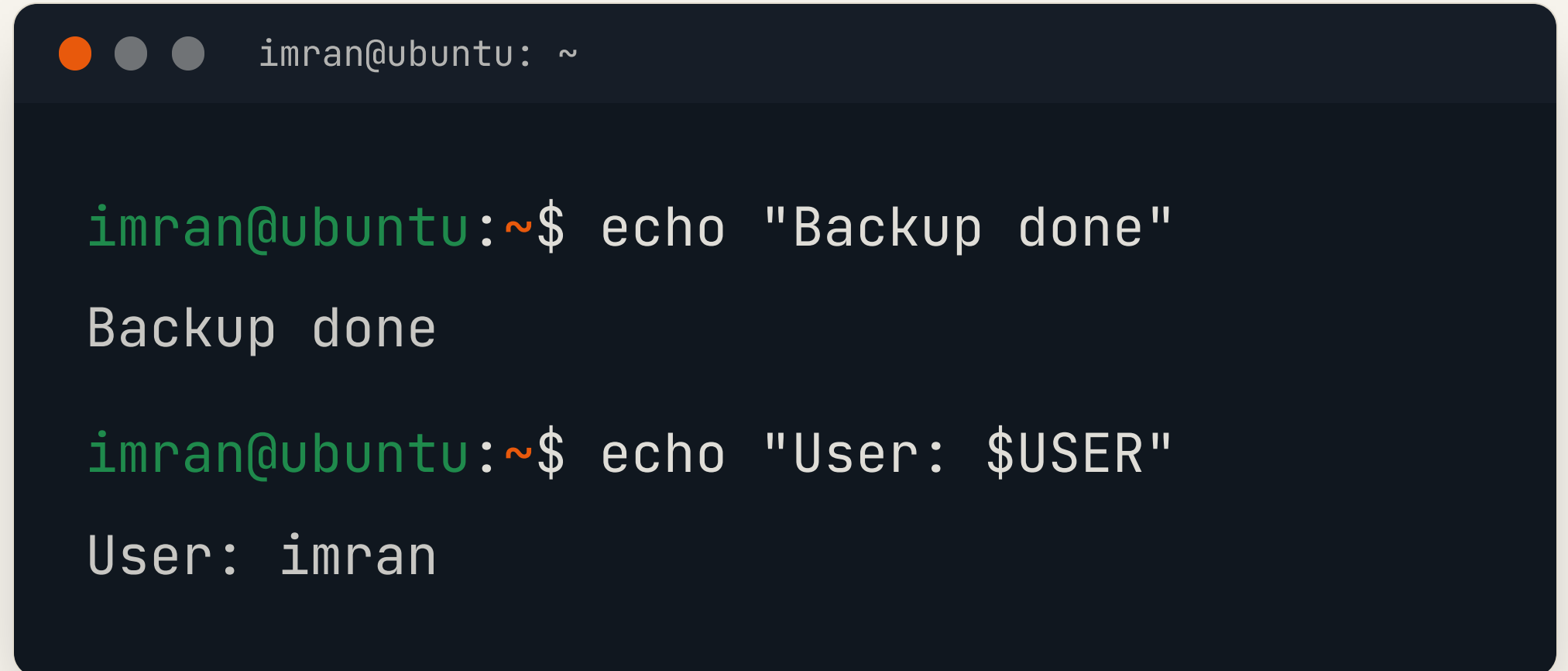
Input and output



PRINTING OUTPUT

echo command

echo prints text and variables to the screen.

A terminal window with a dark blue background and white text. The window title bar shows three colored circles (orange, grey, grey) followed by the text 'imran@ubuntu: ~'. The terminal content shows two commands and their outputs. The first command is 'imran@ubuntu:~\$ echo "Backup done"' followed by the output 'Backup done'. The second command is 'imran@ubuntu:~\$ echo "User: \$USER"' followed by the output 'User: imran'.

```
imran@ubuntu: ~  
  
imran@ubuntu:~$ echo "Backup done"  
Backup done  
  
imran@ubuntu:~$ echo "User: $USER"  
User: imran
```

READING INPUT

read command

read pauses the script and stores what the user types.

ask.sh

```
echo "What is your name?"  
read name  
  
echo "Hello, $name"  
  
# name now holds the typed value
```

PUT IT TOGETHER

Interactive script

Ask, read, then respond using the value the user entered.

greet.sh

```
#!/bin/bash
echo "Enter your city:"
read city
echo "Deploying to $city"

$ ./greet.sh
Enter your city:
Mumbai
Deploying to Mumbai
```


PASSING VALUES IN

Script arguments

```
./script.sh arg1 arg2
```

Arguments let you run the same script with different inputs, no editing needed.

THE SCRIPT NAME

\$0

\$0 holds the name
of the script itself.

```
# inside run.sh
echo "Running $0"

$ ./run.sh
Running ./run.sh
```

FIRST ARGUMENT

\$1

\$1 is the first value you pass after the script name.

```
# inside greet.sh  
echo "Hello $1"  
  
$ ./greet.sh Imran  
Hello Imran
```


SECOND ARGUMENT

\$2

\$2 is the second value. More follow as \$3, \$4, and so on.

```
# inside copy.sh
```

```
echo "From $1 to $2"
```

```
$ ./copy.sh a.txt b.txt
```

```
From a.txt to b.txt
```

ALL ARGUMENTS

\$@

\$@ holds every argument together as a list.

```
# inside list.sh
```

```
echo "Files: $@"
```

```
$ ./list.sh a.txt b.txt c.txt
```

```
Files: a.txt b.txt c.txt
```

HOW MANY

\$#

\$# counts how many arguments were passed.

```
# inside count.sh
echo "You passed $# items"

$ ./count.sh a.txt b.txt
You passed 2 items
```

MAKING DECISIONS

Conditional logic

Is it true?

↓ yes

Run this

↓ no

Skip it

ONE BRANCH

if statement

Run the block only when the test is true. Close with **fi**.

```
if [ $age -ge 18 ]  
then  
    echo "Access granted"  
fi
```


TWO PATHS

if else statement

else runs when the test is false.

```
if [ $age -ge 18 ]  
then  
    echo "Adult"  
else  
    echo "Minor"  
fi
```

MANY PATHS

if elif else

elif checks more conditions in order.

```
if [ $score -ge 90 ]  
then echo "A"  
elif [ $score -ge 60 ]  
then echo "Pass"  
else echo "Retry"  
fi
```

Nested conditions

Logged in?

↳ yes

Is admin?

Dashboard

Home

```
if [ $login = "yes" ]  
then  
  if [ $role = "admin" ]  
  then echo "Dashboard"  
  fi  
fi
```


REPEAT WITHOUT RETYPING

Why loops matter

WITHOUT A LOOP

```
echo file1  
echo file2  
echo file3
```

WITH A LOOP

```
for f in file*  
do echo $f  
done
```

LOOP A KNOWN LIST

for loop



```
for name in Alice Bob Sam
do
    echo "Hi $name"
done
```

LOOP WHILE TRUE

while loop



```
count=1
while [ $count -le 3 ]
do
    echo "Run $count"
    count=$((count+1))
done
```

CONTROL THE LOOP

break and continue

break

Stops the loop completely and moves on.

continue

Skips the rest of this turn, then keeps looping.

PUTTING IT ALL
TOGETHER

Mini automation

A backup script using variables,
a loop, and a condition.

backup.sh

```
#!/bin/bash
src="/var/log"
for file in $src/*.log
do
    if [ -f $file ]
    then cp $file backup/
    fi
done
echo "Backup complete"
```


YOUR TURN

Hands-on exercise

1

Read a name with `read`.

2

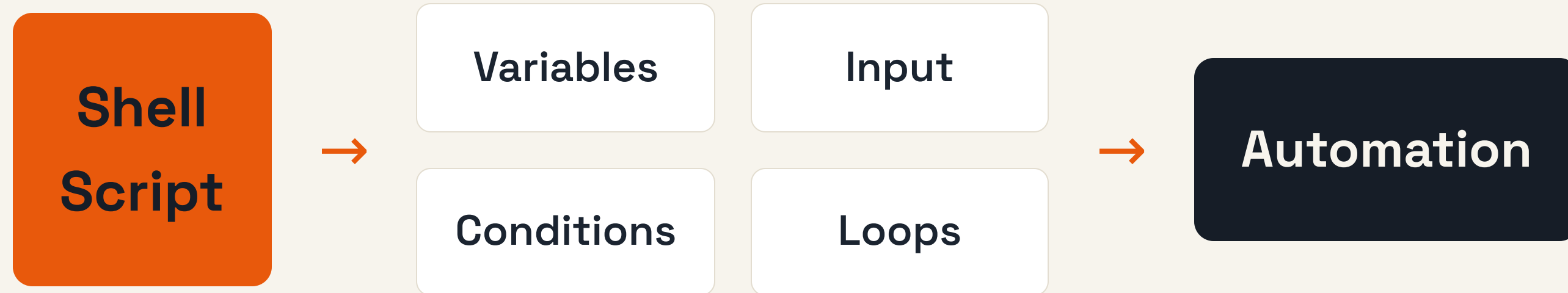
Loop from 1 to that name's count.

3

Print a greeting if the count is even.

THE BIG PICTURE

Revision mind map



CHECK YOURSELF

Quiz & summary

QUESTIONS

What does the shebang line do?

How do you read user input?

When would you use a while loop?

YOU CAN NOW

Write and run Bash scripts

Use variables, input, conditions

Automate tasks with loops